

C Code Example

MPI Send and MPI Recv Examples

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 int main(int argc, char **argv)
7 {
8
9     int rank, size;
10    // Size = 3, in this case
11    /* Start up MPI */
12    MPI_Init(&argc, &argv);
13    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14    MPI_Comm_size(MPI_COMM_WORLD, &size);
15
16
17    // MPI_SEND only copies value to system buf of the destination rank
18    // Data send by one MPI_Send can only be received by one MPI_RECV
19    // if two MPI_SEND calls are made to same rank, then 1 MPI_RECV will recv data
20    // from the first mpi_send, no matter how much you increase the count
21    // MPI_SEND and MPI_RECV work as a couple
22
23    int *sendBuf = (int *)malloc(2 * sizeof(int));
24    sendBuf[0] = 5;
25    sendBuf[1] = 7;
26    int recv[2] = {0, -1};
27
28    if (rank == 0){
29        MPI_Send(sendBuf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);    // Send # 1
30        MPI_Send(sendBuf + 1, 1, MPI_INT, 1, 0, MPI_COMM_WORLD); // Send # 2
31        printf("Both of the data send\n");
32    }
33
34    else if (rank == 1){
35        sleep(5);
36        // The recv count must be >= send count, otherwise it'll throw error
37        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE); // Only recv data from send#1
38        MPI_Recv(recv, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE); // Only recv data from send#2
39
40        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
41    }
42    // MPI recv if the size is more than the data to be received then it ignores (doesn't pick up junk)
43    else if (rank == 2){
44        // this one will be blocked for eternity
45        MPI_Recv(&recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
46        printf("Data received in rank %d, data = %d, %d", rank, recv[0], recv[1]);
47    }
48
49
50    MPI_Finalize();
51
52    return 0;
53 }
```

Listing 1: Example C Code